DAY 2 · CLOSURES & HOF

REAL-LIFE PROJECT

Geeta University · Summer Internship 2026

Project: Shopping Cart Module using Closures & HOF

Project Scenario

Tum ek e-commerce startup ke frontend developer ho.

Task: Ek Shopping Cart module banana hai jo:

• Closure se private state manage kare (cart items, total — direct accessible nahi)

• Higher-order functions use kare for filtering, sorting, transforming cart data

• Currying use kare for discount/tax calculations

• Memoization use kare expensive computations optimize karne ke liye

Ye interview-ready production pattern hai — portfolio mein showcase karne layak.

Part 1 — Core Cart Module (Closure for Private State)

Cart state (items, total) ko closure ke andar private rakhna — bahar se direct access nahi hoga.

```
// cart.js
function createCart(userId) {
// Private state — closure mein band
let items = [];
let userId = userId;
const TAX_RATE = 18;
// Private helper
const computeSubtotal = () =>
items.reduce((sum, item) => sum + item.price * item.qty, 0);
// Public API — ye hi expose karte hain
return {
addItem(product) {
const existing = items.find(i => i.id === product.id);
if (existing) {
items = items.map(i =>
i.id === product.id ? { ...i, qty: i.qty + 1 } : i
);
} else {
items = [...items, { ...product, qty: 1 }];
}
},
removeItem(productId) {
items = items.filter(i => i.id !== productId);
},
```

```
updateQty(productId, qty) {
if (qty <= 0) return this.removeItem(productId);
items = items.map(i =>
i.id === productId ? { ...i, qty } : i
);
},
getItems: () => [...items], // Spread — copy return, original safe
getCount: () => items.reduce((sum, i) => sum + i.qty, 0),
getSubtotal: computeSubtotal,
getTax: () => +(computeSubtotal() * TAX_RATE / 100).toFixed(2),
getTotal:() => +(computeSubtotal() * (1 + TAX_RATE/100)).toFixed(2),
clear: () => { items = []; },
getSummary() {
return {
userId,
itemCount: this.getCount(),
subtotal: this.getSubtotal(),
tax: this.getTax(),
total: this.getTotal(),
};
}
};
}
```

Part 2 — HOF for Cart Analytics

map, filter, reduce use karke cart data transform aur analyze karo.

```
// cartAnalytics.js
const createCartAnalytics = (getItems) => {
// Items by category group karo
const groupByCategory = () =>
getItems().reduce((acc, item) => {
acc[item.category] = acc[item.category] || [];
acc[item.category].push(item);
return acc;
}, {});
// Most expensive item
const getMostExpensive = () =>
getItems().reduce((max, item) =>
item.price > max.price ? item : max
);
// Filter by category
const filterByCategory = category =>
getItems().filter(i => i.category === category);
// Price range filter — curried!
const filterByPriceRange = min => max =>
getItems().filter(i => i.price >= min && i.price <= max);
```

```
// Format for display
const formatItems = () =>
getItems().map(i => ({
display: `${i.name} x${i.qty}`,
lineTotal: `Rs.${(i.price * i.qty).toLocaleString('en-IN')}`,
}));
return { groupByCategory, getMostExpensive,
filterByCategory, filterByPriceRange, formatItems };
};
```

Part 3 — Curried Discount Engine

Discount aur tax calculations ke liye curried functions — reusable aur composable.

```
// discountEngine.js
// Curried discount — apply karo specific user types ko
const applyDiscount = discountPct => subtotal =>
+(subtotal * (1 - discountPct / 100)).toFixed(2);
const studentDiscount = applyDiscount(10); // 10% off
const vipDiscount = applyDiscount(25); // 25% off
const noDiscount = applyDiscount(0);
// Coupon validator + applier
const coupons = {
'INTERN10': applyDiscount(10),
'FLASH50': applyDiscount(50),
'WELCOME15':applyDiscount(15),
};
const applyCoupon = code => subtotal => {
const fn = coupons[code.toUpperCase()];
if (!fn) throw new Error(`Invalid coupon: ${code}`);
return { original: subtotal, discounted: fn(subtotal),
saving: +(subtotal - fn(subtotal)).toFixed(2) };
};
applyCoupon('INTERN10')(5000);
// { original: 5000, discounted: 4500, saving: 500 }
// Compose: discount + tax
const pipe = (...fns) => x => fns.reduce((v, fn) => fn(v), x);
const finalPrice = pipe(
applyDiscount(10),
price => +(price * 1.18).toFixed(2) // add GST
);
finalPrice(1000); // 1062.00 (10% off then 18% GST)
```

Part 4 — Memoized Shipping Calculator

Shipping cost calculation memoize karo — same pincode ke liye dobara API call nahi.

```
// shippingCalculator.js
function memoize(fn) {
```

```
const cache = new Map();
return (...args) => {
const key = JSON.stringify(args);
if (cache.has(key)) return cache.get(key);
const val = fn(...args);
cache.set(key, val);
return val;
};
}
// Simulate expensive pincode lookup
const _getShipping = (pincode, weight) => {
console.log(`Computing for ${pincode}...`); // won't log on cache hit
const zones = { '110': 50, '400': 80, '411': 80, '600': 120 };
const prefix = pincode.toString().slice(0, 3);
const base = zones[prefix] || 150;
return { cost: base + weight * 10, days: base < 100 ? 2 : 4 };
};
const getShipping = memoize(_getShipping);
getShipping(110001, 2); // Computing... { cost: 70, days: 2 }
getShipping(110001, 2); // Cache hit! { cost: 70, days: 2 }
getShipping(400001, 3); // Computing... { cost: 110, days: 4 }
```

Part 5 — Putting It All Together

```
// main.js — Using all modules together
const cart = createCart('user_101');
// Add items
cart.addItem({ id:1, name:'Laptop', price:55000, category:'Electronics' });
cart.addItem({ id:2, name:'T-Shirt', price:499, category:'Clothing' });
cart.addItem({ id:1, name:'Laptop', price:55000, category:'Electronics' });
// id:1 added twice — qty becomes 2
const analytics = createCartAnalytics(cart.getItems);
console.log(cart.getSummary());
// { userId:'user_101', itemCount:3, subtotal:110499, tax:19889.82, total:130388.82 }
console.log(analytics.groupByCategory());
// { Electronics: [Laptop x2], Clothing: [T-Shirt x1] }
const withCoupon = applyCoupon('INTERN10')(cart.getSubtotal());
console.log(withCoupon);
// { original:110499, discounted:99449.10, saving:11049.90 }
console.log(getShipping(110001, 5));
// { cost: 100, days: 2 }
```

Bonus Challenges

1. addItems(...products) banao — rest operator se multiple items ek saath add karo.

2. Cart ko localStorage mein save karo using JSON.stringify/parse.

3. Wishlist module banao same closure pattern se.

4. Quantity limits add karo (max 5 per item) — error handling ke saath.

5. Ek pipe() se: filter(Electronics) → sort(price desc) → map(display format) chain karo.